

LABORATORY OBSERVATION/RECORD

Course: Full Stack Web Development

Course Code: 23CM4121

Branch: CSM

Section: B

Task No.: 5

Student Name: S.Deekshitha

Roll No.: A24126552117

1. TITLE

Build an Interactive Dynamic Webpage Using JavaScript DOM Manipulation

2. AIM

To build an interactive webpage that lets a user pick two colours and generate a live CSS **linear-gradient** preview, with buttons to randomise the colours and copy the generated CSS to the clipboard, using **JavaScript DOM manipulation**.

3. OBJECTIVE

The objectives of this experiment are:

- To use HTML5 color input fields to let the user pick colours visually.
 - To select DOM elements using `document.getElementById()`.
 - To build a `linear-gradient()` CSS string dynamically from user-selected values.
 - To apply a generated style to an element at runtime using `element.style.background`.
 - To display the generated CSS as text using `element.textContent`.
 - To generate random hex colours using `Math.random()` and `toString(16)`.
 - To copy text to the system clipboard using the Clipboard API (`navigator.clipboard`).
 - To give the user temporary visual feedback (e.g. a 'Copied!' label) using `setTimeout()`.
 - To style a clean, centred card-based interface using CSS3.
-

4. THEORY

A CSS **linear-gradient()** blends two or more colours smoothly along a direction (here, left to right). Instead of writing this CSS by hand, this program builds the string in JavaScript from whatever two colours the user currently has selected, then applies it live to a preview box.

The Clipboard API (`navigator.clipboard.writeText()`) lets a webpage copy text directly to the system clipboard without the user needing to manually select and press Ctrl+C, which is why the 'Copy CSS' button works with a single click.

The key methods and concepts used in this program are:

- `<input type="color">` gives the browser's native colour-picker swatch; its `.value` is always a hex string like `#ff6b6b`.
- `document.getElementById()` selects a specific element from the page by its id attribute.
- `element.style.background` sets an element's background directly from JavaScript, used to preview the gradient live.
- `element.textContent` sets the visible text inside an element, used to display the generated CSS line.
- `Math.random()` and `toString(16)` together produce a random number and convert it to a 6-digit hexadecimal colour code.
- `padStart(6, "0")` pads a short hex string with leading zeros so it is always exactly 6 digits long.
- `navigator.clipboard.writeText()` copies a string to the system clipboard, enabling the one-click 'Copy CSS' button.
- `setTimeout(fn, 1500)` restores the button's original label 1.5 seconds after showing 'Copied!'.

The basic flow of the page is:

User picks two colours (or clicks Random) → button click fires → a linear-gradient() string is built → the preview box and CSS text are updated → Copy CSS writes that string to the clipboard

5. ALGORITHM / PROCEDURE

1. Create three files: `index.html`, `style.css`, and `script.js`, and link the CSS and JS files inside the HTML.
 2. Add two `<input type="color">` fields, each with a label and a sensible default value.
 3. Add a 'Generate Gradient' button and a 'Random Gradient' button.
 4. Add an empty preview `<div>` and a `<p>` element to display the generated CSS text.
 5. Add a 'Copy CSS' button next to the CSS text.
 6. In `script.js`, select all of the above elements using `document.getElementById()`.
 7. Define a `generateGradient()` function that reads both colour values and builds a `linear-gradient(...)` string.
 8. Inside `generateGradient()`, apply the string to the preview box's `style.background` and to the CSS text's `textContent`.
 9. Add a click listener on 'Generate Gradient' that calls `generateGradient()`.
 10. Define a `randomColor()` function that builds a random 6-digit hex colour using `Math.random()` and `padStart()`.
 11. Add a click listener on 'Random Gradient' that sets both colour inputs to new random colours, then calls `generateGradient()`.
 12. Add a click listener on 'Copy CSS' that writes the current CSS text to the clipboard using `navigator.clipboard.writeText()`.
 13. Inside that listener, briefly change the button's label to '✅ Copied!' and restore it after 1.5 seconds with `setTimeout()`.
 14. Style the card, buttons, preview box, and code box using CSS3 for a clean, centred layout.
 15. Save the files and open `index.html` in a browser to test all three buttons.
-

6. PROGRAM

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">

  <title>Gradient Generator</title>

  <link rel="stylesheet" href="style.css">
</head>

<body>

  <div class="container">

    <h1> Gradient Generator</h1>
    <p>Create your own beautiful gradient!</p>

    <div class="colors">

      <div>
        <label>Color 1</label>
        <input type="color" id="color1" value="#ff6b6b">
      </div>

      <div>
        <label>Color 2</label>
        <input type="color" id="color2" value="#4ecdc4">
      </div>

    </div>

    <div class="buttons">
      <button id="generateBtn">
        Generate Gradient
      </button>

      <button id="randomBtn">
        Random Gradient
      </button>
    </div>

    <div id="preview"></div>

    <h3>CSS Code</h3>

    <div class="code-box">
      <p id="cssCode">
        linear-gradient(to right, #ff6b6b, #4ecdc4)
```

```

        </p>

        <button id="copyBtn">Copy CSS</button>
    </div>

</div>

<script src="script.js"></script>
</body>
</html>

```

script.js

```

// Get HTML elements using DOM
let color1 = document.getElementById("color1");
let color2 = document.getElementById("color2");

let generateBtn = document.getElementById("generateBtn");
let randomBtn = document.getElementById("randomBtn");

let preview = document.getElementById("preview");
let cssCode = document.getElementById("cssCode");

let copyBtn = document.getElementById("copyBtn");

// Function to generate gradient
function generateGradient() {

    let gradient =
        "linear-gradient(to right, " +
        color1.value + ", " +
        color2.value + ")";

    // Change background using DOM
    preview.style.background = gradient;

    // Display CSS code
    cssCode.textContent = gradient;
}

// Generate gradient when button is clicked
generateBtn.addEventListener("click", function() {

    generateGradient();

});

// Generate random color
function randomColor() {

```

```

    let number = Math.floor(Math.random() * 16777215);

    let color = "#" + number.toString(16).padStart(6, "0");

    return color;
}

// Generate random gradient
randomBtn.addEventListener("click", function() {

    let newColor1 = randomColor();
    let newColor2 = randomColor();

    // Change color inputs
    color1.value = newColor1;
    color2.value = newColor2;

    // Generate gradient
    generateGradient();

});

// Copy CSS code
copyBtn.addEventListener("click", function() {

    navigator.clipboard.writeText(cssCode.textContent);

    copyBtn.textContent = "☑Copied!";

    setTimeout(function() {
        copyBtn.textContent = "Copy CSS";
    }, 1500);

});

```

style.css

```

* {
    box-sizing: border-box;
}

body {
    margin: 0;
    font-family: Arial, sans-serif;
    background-color: #f2f2f2;

    display: flex;
    justify-content: center;
    align-items: center;

```

```
    min-height: 100vh;
}

.container {
  width: 600px;
  background-color: white;

  padding: 30px;
  border-radius: 15px;

  text-align: center;

  box-shadow: 0 5px 20px rgba(0, 0, 0, 0.2);
}

h1 {
  margin-bottom: 5px;
}

p {
  color: #555;
}

.colors {
  display: flex;
  justify-content: center;
  gap: 60px;

  margin: 25px 0;
}

.colors div {
  display: flex;
  flex-direction: column;
  gap: 8px;
}

label {
  font-weight: bold;
}

input[type="color"] {
  width: 100px;
  height: 60px;

  border: none;
  cursor: pointer;
}

.buttons {
  display: flex;
  justify-content: center;
}
```

```
    gap: 10px;

    margin-bottom: 20px;
}

button {
    padding: 12px 18px;

    border: none;
    border-radius: 8px;

    background-color: #333;
    color: white;

    cursor: pointer;
}

button:hover {
    opacity: 0.8;
}

#preview {
    width: 100%;
    height: 220px;

    border-radius: 12px;

    background: linear-gradient(
        to right,
        #ff6b6b,
        #4ecdc4
    );

    margin-bottom: 25px;
}

h3 {
    margin-bottom: 10px;
}

.code-box {
    background-color: #eeeeee;

    padding: 15px;

    border-radius: 8px;
}

#cssCode {
    font-family: monospace;
    color: #333;

    word-break: break-all;
```

```

}

#copyBtn {
  background-color: #007bff;
}

```

7. CODE EXPLANATION

Code	Explanation
<code><input type="color"></code>	Gives a native colour-picker swatch whose .value is a hex string like #ff6b6b.
<code>document.getElementById()</code>	Selects a specific element from the page by its id attribute.
<code>generateGradient()</code>	Reads both colour values, builds a linear-gradient() string, and applies it to the preview and CSS text.
<code>preview.style.background</code>	Sets the preview box's background directly from JavaScript.
<code>cssCode.textContent</code>	Displays the generated CSS string as visible text.
<code>Math.random() * 16777215</code>	Produces a random number up to the largest possible 24-bit colour value (0xFFFFFFFF).
<code>number.toString(16)</code>	Converts that number into a hexadecimal string, the format hex colours use.
<code>padStart(6, "0")</code>	Pads the hex string with leading zeros so it is always a valid 6-digit colour code.
<code>navigator.clipboard.writeText()</code>	Copies the given text directly to the system clipboard.
<code>setTimeout(fn, 1500)</code>	Runs a function once, 1.5 seconds later — used to reset the Copy button's label.
<code>box-shadow, border-radius</code>	Give the card and preview box their soft, rounded, elevated appearance.

8. TEST CASES AND EXECUTION DATA

The page was opened in Chrome and each control was tested individually; the results were recorded below.

Test Case	Action	Expected Result	Actual Result	Status
TC-01	Open index.html in a browser	Page loads with the default red-to-teal gradient preview and matching CSS text	Page displayed correctly	Pass
TC-02	Pick two new colours and click 'Generate Gradient'	Preview box and CSS text update to the new colours	Preview and CSS updated correctly	Pass
TC-03	Click 'Random Gradient' several times	Both colour swatches and the preview change to a new random pair each time	Random colours applied correctly	Pass
TC-04	Click 'Copy CSS'	Button label briefly changes to '✅ Copied!' and the CSS text is copied to the clipboard	Label changed and clipboard updated correctly	Pass
TC-05	Wait 1.5 seconds after copying	Button label reverts to '📄 Copy CSS'	Label reverted correctly	Pass
TC-06	Hover over any button	Button becomes slightly more transparent (opacity change)	Hover effect displayed	Pass

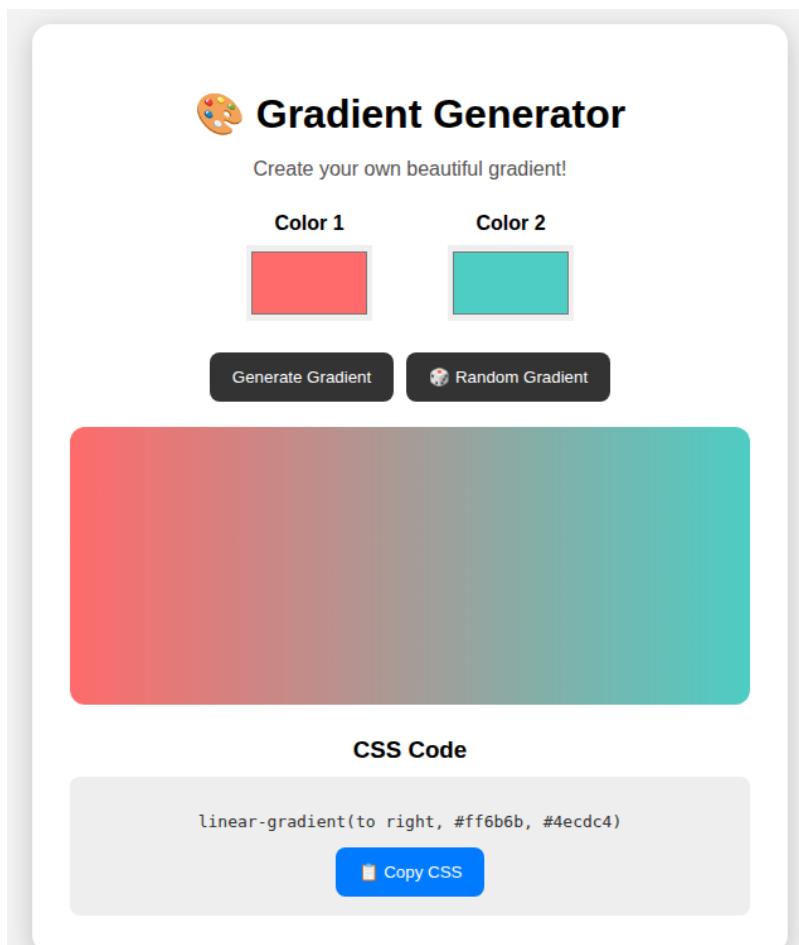
Additional Validation Test

Test Case	Input/Action	Expected Result	Status
TC-07	Run the page in a Safari / WebKit-based browser	Page and buttons should work exactly as in Chrome	Fail – some WebKit engines throw 'Can't create duplicate variable that shadows a global'

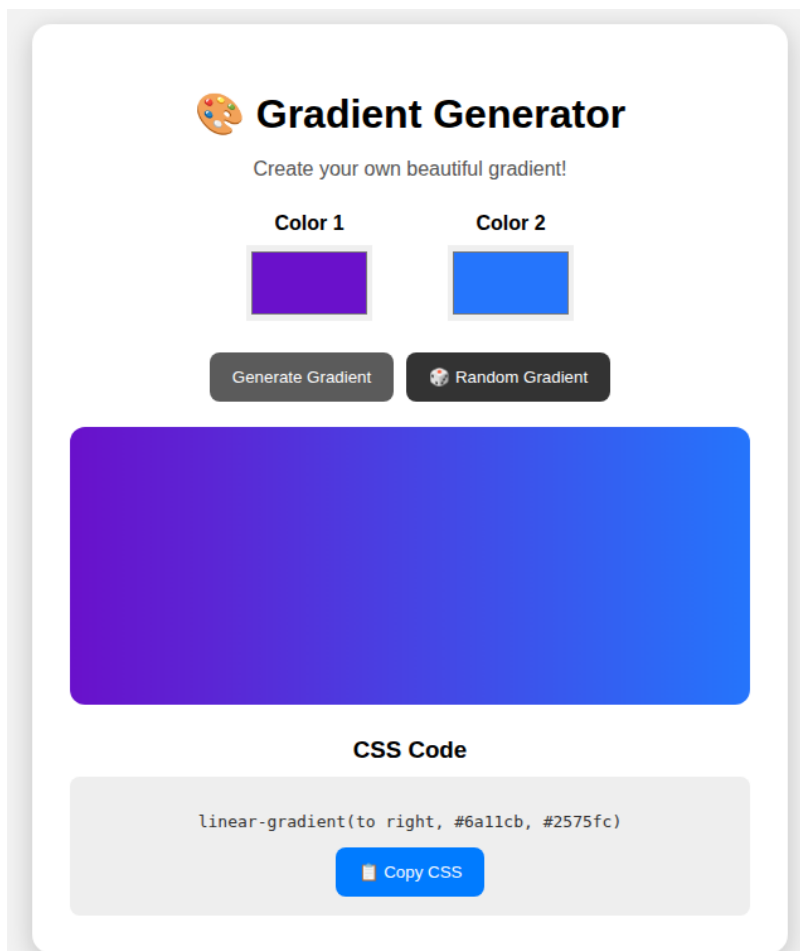
			property' because let color1, let randomBtn, etc. reuse the same names as each element's id attribute (elements with an id are auto-exposed as global bindings)
--	--	--	---

9. OUTPUT

Output 1: Default Gradient on Page Load



Output 2: Custom Colours after 'Generate Gradient'



Output 3: After Clicking 'Random Gradient'



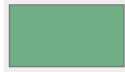
Gradient Generator

Create your own beautiful gradient!

Color 1



Color 2



Generate Gradient

Random Gradient



CSS Code

```
linear-gradient(to right, #951d38, #70ae85)
```

Copy CSS

Output 4: After Clicking 'Copy CSS'



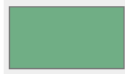
Gradient Generator

Create your own beautiful gradient!

Color 1



Color 2



Generate Gradient

Random Gradient



CSS Code

```
linear-gradient(to right, #951d38, #70ae85)
```

✓ Copied!

10. RESULT

Thus, an interactive CSS gradient generator was successfully built using HTML colour inputs and JavaScript DOM manipulation. Generating, randomising, and copying gradients all worked correctly when tested in Chrome. One cross-browser compatibility issue was observed: naming top-level let variables the same as element id attributes (e.g. `let color1`, `let randomBtn`) causes a `SyntaxError` in Safari/WebKit-based browsers, because elements with an id are automatically exposed as global bindings that `let/const` cannot redeclare; renaming these variables (e.g. `color1Input`, `randomBtnEl`) would make the script portable across all browsers.